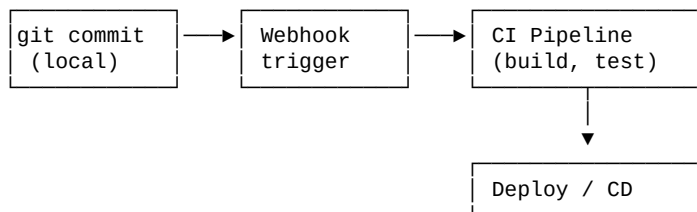


VERSION CONTROL & BRANCHING STRATEGIES FOR CI

1. Foundation: Where the Pipeline Starts

The CI/CD pipeline begins the moment a developer commits code to a version control system (Git). The way commits are structured, branched, and merged directly affects build, test, and deployment processes. Choosing the right branching strategy and repository structure is critical for efficient automation.

Development to Pipeline Flow:



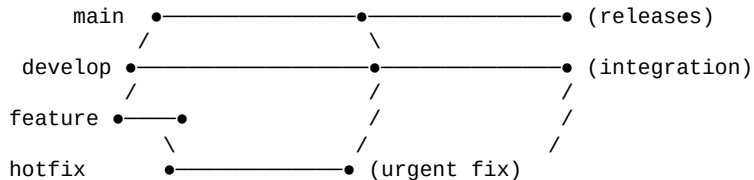
2. Branching Strategies Overview

Branching strategies define how developers collaborate, integrate changes, and release software. Two predominant models: **Git Flow** (long-lived branches) and **Trunk-Based Development** (short-lived branches, mainline integration).

3. Git Flow

Created by Vincent Driessen. Uses multiple long-lived branches: **main** (production), **develop** (integration), and supporting branches: **feature/***, **release/***, **hotfix/***.

Git Flow Branch Model:



- **main** – production-ready code. Only merges from develop or hotfix.
- **develop** – integration branch for features. Contains latest delivered changes.
- **feature/*** – created from develop, merged back after completion.
- **release/*** – preparation for a release (bug fixes, version bumps). Merged into main and develop.
- **hotfix/*** – emergency fixes from main, merged into both main and develop.

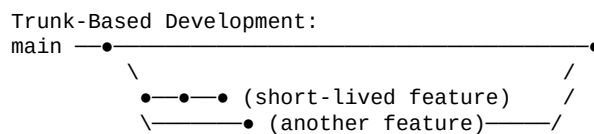
```
# Feature branch workflow (Git Flow)
git checkout develop
git checkout -b feature/new-login
# ... work, commits ...
git checkout develop
git merge --no-ff feature/new-login

# Release branch
git checkout -b release/1.2.0 develop
# final adjustments, then:
git checkout main
git merge release/1.2.0
git tag -a 1.2.0
git checkout develop
git merge release/1.2.0
```

When to use Git Flow: Projects with scheduled releases, multiple versions in maintenance, large teams preferring structured release cycles. But it adds complexity; not ideal for continuous delivery.

4. Trunk-Based Development (TBD)

Developers merge small, frequent changes into a single branch (trunk, main or master) multiple times a day. Short-lived feature branches (hours or days).



- No long-running feature branches. Branches (if any) live < 1 day.
- Use feature toggles (flags) to hide incomplete features.
- Relies on automated tests and CI to catch integration issues.
- Enables true continuous delivery: every commit can be releasable.

```
# Trunk-based workflow
git checkout main
git pull --rebase
git checkout -b short-feature
# ... small change, commit ...
git push -u origin short-feature
# Create PR, get review, merge to main (no merge commit if squash/rebase)
git checkout main
git pull
git merge --squash short-feature
git push
```

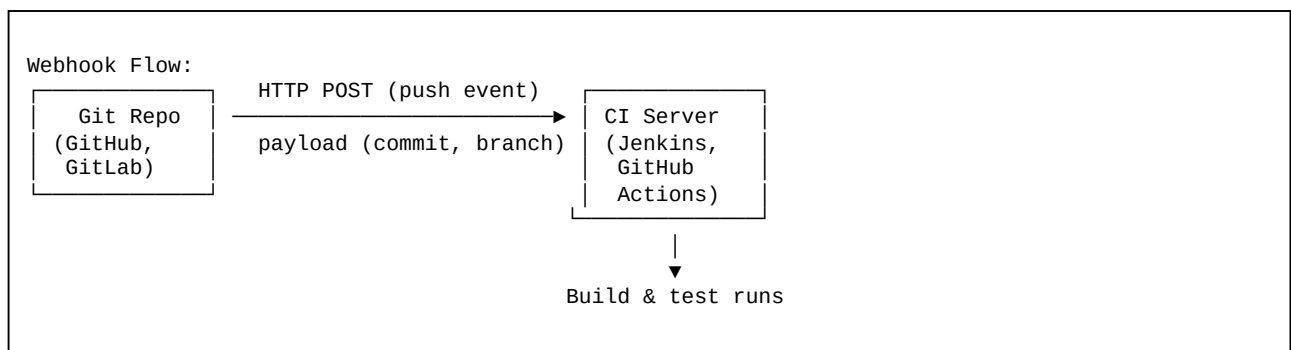
When to use TBD: CI/CD mature teams, need fast feedback, microservices, daily deployments. Most common in modern DevOps cultures.

4.1 Git Flow vs Trunk-Based Comparison

Aspect	Git Flow	Trunk-Based Development
Release frequency	Scheduled (e.g., weekly/monthly)	Continuous (multiple per day)
Branch lifetime	Days to weeks (feature, release)	Hours (or no branches)
Merge complexity	Potential merge hell (long-lived branches)	Low (small, frequent merges)
Integration pain	Late integration, painful conflict resolution	Early and continuous integration
Suitable for	Large repositories, multiple maintained versions	Microservices, cloud-native, CD
Manual testing burden	Higher, release branches need validation	Lower, automated tests catch issues quickly
Use of feature flags	Optional, but often not enforced	Essential for hiding work-in-progress

5. Webhooks – Automating Pipeline Triggers

Webhooks are HTTP callbacks triggered by Git events (push, pull request, tag creation). CI/CD systems listen for webhooks to start pipelines automatically.



```
# Example: Configuring a webhook in GitHub (using GitHub CLI or UI)
gh repo webhook create --repo myorg/myrepo --url https://ci.example.com/webhook --events
push,pull_request
```

```
# Payload includes: ref (branch), commits, head_commit, etc. (used to set pipeline
parameters)
```

```
# Sample payload processing in CI (pseudo-code)
if (event == "push" && ref == "refs/heads/main") {
  run_pipeline("deploy-to-staging");
} else if (event == "pull_request") {
  run_pipeline("validate-pr");
}
```

Security: Use secret tokens to verify webhook authenticity; validate the signature header (e.g., X-Hub-Signature-256).

```
# GitHub webhook secret verification (Node.js example)
const crypto = require('crypto');
const signature = req.headers['x-hub-signature-256'];
const computed = 'sha256=' + crypto.createHmac('sha256',
SECRET).update(JSON.stringify(req.body)).digest('hex');
if (signature !== computed) { return res.status(401).send('Invalid signature'); }
```

6. Monorepo vs Polyrepo (Multi-repo)

Repository structure affects pipeline design, build triggering, and dependency management.

6.1 Monorepo (Single Repository)

All code of an organization (or major product) in one repository; multiple applications, services, libraries.

Monorepo Structure:

```
monorepo/
├── apps/
│   ├── web-app/
│   ├── mobile-app/
│   └── backend-service/
├── libs/
│   ├── shared-utils/
│   └── auth-lib/
├── tools/ (CI configs, scripts)
└── package.json (or build config root)
```

- **Pros:** Single source of truth, atomic cross-service commits, easier code reuse, simplified dependency management (one version).
- **Cons:** Repository can become huge (size, clone time); pipeline must handle selective builds (only changed components); complex tooling (Bazel, Nx, Gradle).
- **CI strategies:** Use affected/dependency graph to build only changed projects + their dependents (e.g., Nx affected:build, git diff to detect).

6.2 Polyrepo (Multiple Small Repositories)

Each microservice, library, or application has its own Git repository.

```
Polyrepo Structure:  
repo-1 (web-app)  
repo-2 (auth-service)  
repo-3 (shared-lib)  
repo-4 (mobile-api)
```

- **Pros:** Clear ownership, independent versioning and CI pipelines, fast clone/fetch, scalability for large teams.
- **Cons:** Cross-repo changes are difficult (multiple PRs, coordination), versioning hell (shared libs need semantic versioning and package registries), cannot atomically change multiple services.
- **CI strategies:** Each repo has its own pipeline, triggered by pushes. Use CI artifacts/registries to share libraries (NPM, Maven, Docker).

6.3 Monorepo vs Polyrepo Comparison Table

Factor	Monorepo	Polyrepo
Code sharing	Direct (same repo, no version bumps)	Via package managers (versioned)
Cross-service changes	Single commit, atomic	Multiple pull requests, coordination

Repository size	Grows large (can be TBs)	Small per repo
CI pipeline complexity	Need selective building & caching	Per-repo pipelines, simpler logic
Dependency management	Single version, no duplication	Possible version conflicts (DLL hell)
Team autonomy	Requires coordination & conventions	High (own repo = own process)
Tooling	Bazel, Nx, Turborepo, Lerna	Standard CI per repo

Recommendation:

- Use **Monorepo** if your organization practices agile cross-team development, needs atomic changes, or has many interdependent services.
- Use **Polyrepo** for isolated services, open-source projects, or when independent versioning is critical.
- Many large orgs start with polyrepo and eventually move to monorepo (or use hybrid: shared libs monorepo + microservice polyrepo).

7. Putting It All Together – CI Pipeline Triggering

```
# Example GitHub Actions workflow with branch filters and monorepo path filtering
name: CI Pipeline
on:
  push:
    branches: [ main, develop, release/** ]
    paths:
      - 'apps/web-app/**'
      - 'libs/shared/**'
  pull_request:
    branches: [ main ]
    types: [ opened, synchronize ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # needed for impacted detection
      - name: Detect changes (monorepo)
        run: |
          git diff --name-only ${github.event.before} ${github.sha} | tee
          changed.txt
      - name: Build affected projects (Nx example)
        run: npx nx affected --target=build --base=${github.event.before}
```

Key Takeaways:

1. The moment code is committed, webhooks start the pipeline – so branch hygiene directly affects release speed.
 2. Trunk-Based Development is preferred for CI/CD; Git Flow works for periodic releases but creates more overhead.
 3. Webhooks enable decoupled, event-driven automation; always secure them with secrets.
 4. Choose monorepo or polyrepo based on team size, tooling, and coupling between services. Both are viable.
-